

DivideAndConcur: Probe-Guided Systematic Interleaving Testing for Java

Jayanta Majumder, Sambuddha Majumder

May 10, 2026

Abstract

Concurrent programs often fail only under rare interleavings. Conventional unit tests usually leave the relevant schedule to chance, while full formal modelling requires a separate specification language and substantial abstraction work. This paper presents DivideAndConcur, abbreviated DnC, a lightweight Java prototype for probe-guided deterministic concurrency testing. A developer marks semantically meaningful scheduling points in ordinary Java code using calls such as `init`, `barrier`, and `end`. DnC treats the code between adjacent probes as an atomic segment and executes these segments under generated or prescribed schedules. The approach is inspired by labelled atomic actions in PlusCal/TLA+ and by systematic concurrency testing, but it deliberately aims at a pragmatic middle ground: executable Java code, explicit interleaving control, reproducible failure schedules, and examples that can be understood directly from the original code. The paper describes the design, execution semantics, schedule generation, failure reporting, and several worked examples: lost update, lazy initialisation, constrained schedule exploration, and a bounded Lamport bakery-style mutual exclusion sketch.

1 Motivation

Concurrency defects are often not caused by the individual statements of a procedure, but by the order in which statements from different threads are interleaved. A method may pass thousands of ordinary unit-test runs because the operating system scheduler rarely produces the problematic ordering. When the defect finally appears in production, the observed failure can be hard to reproduce because the schedule was not recorded as an input to the test.

DivideAndConcur addresses this by turning the schedule into a first-class test artefact. Instead of asking whether a program passes under whatever schedule the runtime happens to choose, DnC asks a more precise question:

Given these semantic scheduling points, does the code satisfy its invariants under all generated interleavings, or under this particular prescribed interleaving?

The tool is intentionally situated between two extremes. At one extreme, a stress test repeatedly runs real code and hopes to observe a failure. At the other extreme, a formal model checker explores a mathematical model written in a dedicated specification language. DnC keeps the code executable and Java-like, but gives the test harness deterministic control over the order of semantically atomic regions.

2 Relationship to PlusCal and Systematic Concurrency Testing

PlusCal (introduced in [4]) allows an algorithm to be written with labels; code between labels is translated into atomic actions in a TLA+ specification. DnC borrows this labelled-action idea, but the representation remains Java code. A developer can copy or adapt a concurrency-critical procedure into a test and insert probes where context switches are meaningful.

DnC also belongs to the broader family of systematic concurrency testing and stateless model checking. Systems such as CHESS demonstrated that many concurrency bugs can be found by controlling and replaying thread schedules rather than merely relying on stress testing [1]. DnC is not a reimplementaion of CHESS. It is smaller and more explicit: the developer supplies semantic probe points, and DnC explores interleavings of those probe-delimited segments.

3 Programming Model

The DnC API is deliberately small. A logical thread starts its controlled region with `init`, separates atomic segments with `barrier`, and finishes with `end`.

Listing 1: DnC-instrumented code shape

```
gate.init("t1");  
// segment 0  
  
gate.barrier("t1");  
// segment 1  
  
gate.barrier("t1");  
// segment 2  
  
gate.end("t1");
```

The logical thread id, for example `"t1"`, need not be the actual Java thread name. It is the name used by the scheduler. The important design improvement in the current version is that the segment index is not written in the instrumented code. Instead, the gate maintains a per-thread counter internally. This keeps the inserted probes lightweight and reduces opportunities for numbering mistakes.

A schedule, however, still refers to explicit turns:

Listing 2: A prescribed schedule

```
List.of(  
    Turn.of("t1", 0),  
    Turn.of("t2", 0),  
    Turn.of("t1", 1),  
    Turn.of("t2", 1)  
);
```

This schedule means that `t1` executes segment 0, then `t2` executes segment 0, then `t1` executes segment 1, and finally `t2` executes segment 1.

4 Core Semantics

A key correctness rule is that the schedule advances when a segment completes, not when it starts. If the scheduler advanced merely because a thread passed an entry barrier, the next thread might start

before the current thread had completed the supposedly atomic region. DnC therefore maintains a single currently running logical thread.

The gate state includes:

- the prescribed schedule;
- the current schedule index;
- a map from logical thread id to current segment counter;
- the currently running logical thread, if any;
- the currently running turn;
- a trace of segment starts and completions;
- optional invariants checked after each segment.

The lifecycle is as follows:

1. `init(tid)` waits until `Turn.of(tid, 0)` is the next scheduled turn and starts segment 0.
2. `barrier(tid)` completes the current segment, checks invariants, increments the thread's segment counter, then waits until the next segment for that thread is scheduled.
3. `end(tid)` completes the current segment, checks invariants, and marks the controlled region finished for that thread.

This gives a simple but strong guarantee: at most one scheduled segment is executing at a time.

5 Schedule Generation

Given per-thread segment counts, DnC generates order-preserving interleavings. For example:

Listing 3: High-level schedule specification

```
Map.of(
  "t1", 2,
  "t2", 2
)
```

means that the generated schedules should contain `t1:0`, `t1:1`, `t2:0`, and `t2:1`, while preserving each thread's internal order:

$$\begin{aligned} t1 : 0 < t1 : 1 \\ t2 : 0 < t2 : 1 \end{aligned}$$

For two threads with two segments each, there are six schedules:

Listing 4: All two-by-two interleavings

```
[t1:0, t1:1, t2:0, t2:1]
[t1:0, t2:0, t1:1, t2:1]
[t1:0, t2:0, t2:1, t1:1]
[t2:0, t1:0, t1:1, t2:1]
[t2:0, t1:0, t2:1, t1:1]
[t2:0, t2:1, t1:0, t1:1]
```

In general, for threads with segment counts $n_1, n_2, \dots, n_k$, the unconstrained number of schedules is:

$$\frac{(n_1 + n_2 + \dots + n_k)!}{n_1! n_2! \dots n_k!}.$$

This grows quickly. DnC therefore uses a lazy visitor-style generator so schedules can be executed one at a time without materialising the entire space. It also supports basic constraints:

Listing 5: Constrained schedule generation

```
ScheduleOptions.exhaustive()
    .requireBefore(Turn.of("t1", 0), Turn.of("t2", 1))
    .maxPreemptions(1);
```

A `BeforeConstraint` can rule out impossible or uninteresting schedules, while a `preemption` bound prioritises schedules that switch away from a still-runnable thread only a limited number of times.

6 The Example Suite

The prototype contains an executable example suite. The suite is deliberately chosen to demonstrate several categories of bug and non-bug:

Example	Purpose	Expected outcome
Lost update	Demonstrates a classic read-modify-write race	Some schedules fail
Lazy initialisation	Demonstrates check-then-act race	Some schedules fail
Constrained schedule	Demonstrates generation constraints and preemption bounding	Generated schedules pass
Bakery sketch, correct tie-breaker	Bounded mutual exclusion sketch	All explored schedules pass
Broken bakery, no tie-breaker	Demonstrates a mutual exclusion violation	Some schedules fail

The output observed from the example suite is summarised below:

Listing 6: Example suite summary

```
Lost update:
runs=6, passed=2, failed=4

Lazy init bug:
runs=6, passed=2, failed=4

Constrained schedule:
runs=3, passed=3, failed=0

Bakery sketch, correct tie-breaker:
runs=924, passed=924, failed=0

Broken bakery, no tie-breaker:
runs=70, passed=46, failed=24
```

The next sections discuss these examples in more detail.

7 Example 1: Lost Update

The lost-update example models a non-atomic increment. The original operation is conceptually simple: read a shared counter, add one, and write it back. The DnC-instrumented procedure is:

Listing 7: Lost update procedure

```
void increment(String tid, ScheduleGate gate) {
    gate.init(tid);
    local = counter.value;          // segment 0: read shared state

    gate.barrier(tid);
    counter.value = local + 1;      // segment 1: write shared state

    gate.end(tid);
}
```

The final assertion requires the counter to be 2 after two increments:

```
Assertions.check(counter.value == 2,
    "lost update: expected counter=2, observed counter=" + counter.value);
```

DnC explores all six two-by-two interleavings. Four fail. The canonical failing schedule is:

```
[t1:0, t2:0, t1:1, t2:1]
```

This corresponds to:

1. t1 reads `counter.value == 0`;
2. t2 reads `counter.value == 0`;
3. t1 writes 1;
4. t2 writes 1.

The final value is 1, not 2. DnC's report is useful because it names the exact schedule, not merely the final symptom.

8 Example 2: Lazy Initialisation Race

The second example demonstrates a check-then-act bug. The suspicious pattern is:

```
if (!registry.states.containsKey("IBM")) {
    registry.constructed++;
    registry.states.put("IBM", new Object());
}
```

In the DnC version, the check and act are placed in separate segments:

Listing 8: Lazy initialisation race

```
void getOrCreate(String tid, ScheduleGate gate) {
    gate.init(tid);
    missing = !registry.states.containsKey("IBM");    // segment 0

    gate.barrier(tid);
    if (missing) {                                     // segment 1
        registry.constructed++;
        registry.states.put("IBM", new Object());
    }
}
```

```

    gate.end(tid);
}

```

The final assertion requires exactly one construction:

```

Assertions.check(registry.constructed == 1,
    "expected exactly one construction, observed " + registry.constructed);

```

A failing schedule is:

```

[t1:0, t2:0, t1:1, t2:1]

```

Both threads observe the entry as missing before either constructs it. Both then construct. This is exactly the kind of semantic interleaving that can be missed by ordinary tests but becomes obvious when the schedule is controlled.

A repair in real Java might use a synchronised block or `ConcurrentHashMap.computeIfAbsent`. DnC's value is that it gives a small, deterministic witness for the bug before and after the repair.

9 Example 3: Constrained Schedule Exploration

Not every test needs the full schedule space. DnC supports constraints that cut the space. The constrained example uses:

```

ScheduleOptions options = ScheduleOptions.exhaustive()
    .requireBefore(Turn.of("t1", 0), Turn.of("t2", 1))
    .maxPreemptions(1);

```

For two threads with two segments each, the full space contains six schedules. These constraints reduce the explored space to three schedules in the included example. This is not yet full partial-order reduction, but it demonstrates the design direction: the schedule generator can be progressively refined with domain knowledge.

10 Example 4: Lamport Bakery-Style Mutual Exclusion Sketch

The fourth example is a bounded sketch inspired by Lamport's bakery algorithm [2]. The aim is not to prove the full algorithm, including liveness and the choosing flag protocol. Instead, it demonstrates that DnC can exercise a mutual-exclusion-style safety property over interleavings.

The correct sketch uses lexicographic ticket ordering:

```

private boolean less(int idA, int ticketA, int idB, int ticketB) {
    return ticketA < ticketB || (ticketA == ticketB && idA < idB);
}

```

Each worker performs three segments:

1. take a ticket;
2. compare tickets and possibly enter the critical section;
3. leave the critical section and clear the ticket.

The critical section assertion is:

```

s.inCritical++;
Assertions.check(s.inCritical <= 1,
    "mutual exclusion violated in correct bakery sketch");

```

The example explores 924 schedules and finds no violation. This is a useful demonstration of a bounded safety check, but it should not be overstated. DnC is not proving the general bakery algorithm for all executions. It is testing a bounded executable scenario.

11 Example 5: Broken Bakery Without Tie-Breaker

The final example deliberately removes the process-id tie-breaker. The broken condition is:

```
boolean mayEnter = s.number[other] == 0 ||  
    s.number[id] <= s.number[other]; // BUG: no id tie-breaker
```

If both workers choose ticket 1, each may believe it can enter. DnC finds this mutual exclusion violation in 24 of the explored schedules:

```
Broken bakery, no tie-breaker:  
runs=70, passed=46, failed=24
```

This example illustrates the kind of reasoning that DnC can make concrete. The algorithmic defect is not merely stated abstractly; it is exhibited as an executable interleaving.

12 Failure Reports as Debugging Artefacts

A useful concurrency-testing tool should not simply report that a race exists. It should give a reproducible witness. DnC's `RunResult.pretty()` method prints:

- whether the run passed or failed;
- the schedule;
- the exception or failed assertion;
- the trace of started and completed segments.

A schedule such as:

```
[t1:0, t2:0, t1:1, t2:1]
```

is both human-readable and machine-replayable. This is important because many concurrency failures are not useful unless they can be reproduced.

13 Design Trade-Offs

13.1 Manual Probes Rather Than Automatic Instrumentation

DnC intentionally uses manual probes. This gives the developer semantic control. The tool does not guess where a context switch matters; the developer marks those points.

The cost is modelling discipline. If relevant shared-state effects happen inside a large segment, DnC will treat them as atomic and may miss bugs that require interleaving inside that segment.

13.2 Real Java Threads, Controlled Logical Schedule

The implementation uses real Java threads, but only one logical thread is allowed to run a DnC segment at a time. This gives compatibility with ordinary Java code while avoiding dependence on the OS scheduler for the interleaving under test.

13.3 Interleaving Testing, Not Memory-Model Verification

DnC is not a Java Memory Model verifier. It does not enumerate compiler, CPU, volatile, fence, or relaxed-memory behaviours. Tools such as JVM stress harnesses are more appropriate for low-level memory-model questions. DnC is aimed at higher-level semantic interleavings.

14 Limitations

The current prototype has several limitations:

- no state snapshot or state graph export yet;
- no read/write-set-based partial-order reduction;
- no automatic probe insertion;
- no JUnit extension wrapper yet;
- no schedule minimisation after failure;
- no integration with Java Memory Model exploration;
- no symbolic execution or data generation.

These limitations are acceptable for a prototype because the central claim is narrower: semantically labelled Java code can be run under deterministic, generated interleavings with reproducible failure schedules.

15 Future Work

The most immediate next steps are:

1. add JSON input/output for schedules and traces;
2. add Graphviz DOT output for schedule traces and state graphs;
3. add state snapshot hooks;
4. add schedule minimisation for failing traces;
5. add read/write declarations for partial-order reduction;
6. add a JUnit 5 extension;
7. add repaired versions of the examples;
8. add a PlusCal/TLA+ export experiment.

The longer-term research direction is to combine host-language executable tests with model-checking-inspired reductions. A useful intermediate target would be a system that can explore schedules exhaustively for small critical sections, prune equivalent schedules using read/write independence, and export failing traces as documentation or formal-model seeds.

16 Conclusion

DivideAndConcur is a lightweight experiment in making concurrency schedules explicit in Java tests. It borrows the labelled atomic-region idea from PlusCal and the reproducible schedule exploration idea from systematic concurrency testing, but keeps the developer experience close to ordinary code. The examples show that even a small prototype can expose classic races such as lost updates and lazy initialisation bugs, while also supporting bounded mutual exclusion sketches. The central value of the tool is not merely that it finds failures, but that it explains them as concrete interleavings.

References

- [1] Madanlal Musuvathi, Shaz Qadeer, Thomas Ball, Gerard Basler, Piramanayagam Arumuga Nainar, and Iulian Neamtiu. Finding and Reproducing Heisenbugs in Concurrent Programs. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation*, 2008.
- [2] Leslie Lamport. A New Solution of Dijkstra’s Concurrent Programming Problem. *Communications of the ACM*, 17(8):453–455, 1974.
- [3] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.
- [4] Leslie Lamport. The PlusCal algorithm language. In *International Colloquium on Theoretical Aspects of Computing*, pages 36–60. Springer, 2009.